



Guía: Crear un proyecto "Hola Mundo" con Spring Boot y NetBeans – parte 2

Introducción

En esta lección aprenderás cómo agregar un controlador REST básico a tu proyecto `hola-spring` creado en NetBeans, y cómo personalizar configuraciones usando el archivo `application.properties`.

Paso 1: Crear el Controlador REST

1. En NetBeans, haz clic derecho sobre el paquete `gm.holaspring`
2. Selecciona:

New > Java Class

3. Escribe como nombre de la clase:

ControladorInicio

4. Elimina los comentarios de la clase (Ctrl+E) para limpiar el archivo.

Paso 2: Configurar `application.properties`

Abre el archivo:

`src/main/resources/application.properties`

Agrega las siguientes configuraciones:

```
#server.port=9090
logging.level.gm=DEBUG
logging.pattern.dateformat=hh:mm
spring.main.banner-mode=off
```

¿Qué logras con esto?

- El servidor Tomcat se ejecutará en el puerto **9090** en caso de que habilites esta configuración.
- Verás mensajes más detallados en la consola para el paquete `gm`.
- Los logs mostrarán solo la **hora y minutos**.
- Deshabilitarás el logo de Spring al iniciar la aplicación haciendo más limpia la salida a consola

Paso 3: Agregar Anotaciones y Método

En la clase `ControladorInicio.java`, escribe lo siguiente:

```
package gm;

import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RestController;

@RestController
public class ControladorInicio {
```

```
private static final Logger log = LoggerFactory.getLogger(ControladorInicio.class);

@GetMapping("/")
public String inicio() {
    log.info("ejecutando el controlador rest");
    log.debug("mas detalle del controlador");
    return "HolaMundo con Spring Boot";
}
}
```

Explicaciones importantes:

- `@RestController`: Indica que esta clase maneja peticiones web.
- `@GetMapping("/")`: Asocia el método `inicio()` a la ruta principal del navegador (<http://localhost:8080>).
- Creamos la variable `log` para el manejo de mensajes de tipo `Slf4j` a consola



Paso 4: Ejecutar la Aplicación

1. Haz clic derecho sobre `HolaSpringApplication.java`.
2. Selecciona `Run File`.

NetBeans levantará automáticamente el servidor **Tomcat embebido** en el puerto 8080.



Paso 4: Verificar en el Navegador

1. Abre tu navegador y escribe:

```
http://localhost:8080
```

2. Verás el mensaje:

```
Hola Mundo con Spring Boot
```

3. También en la consola de NetBeans verás los mensajes:

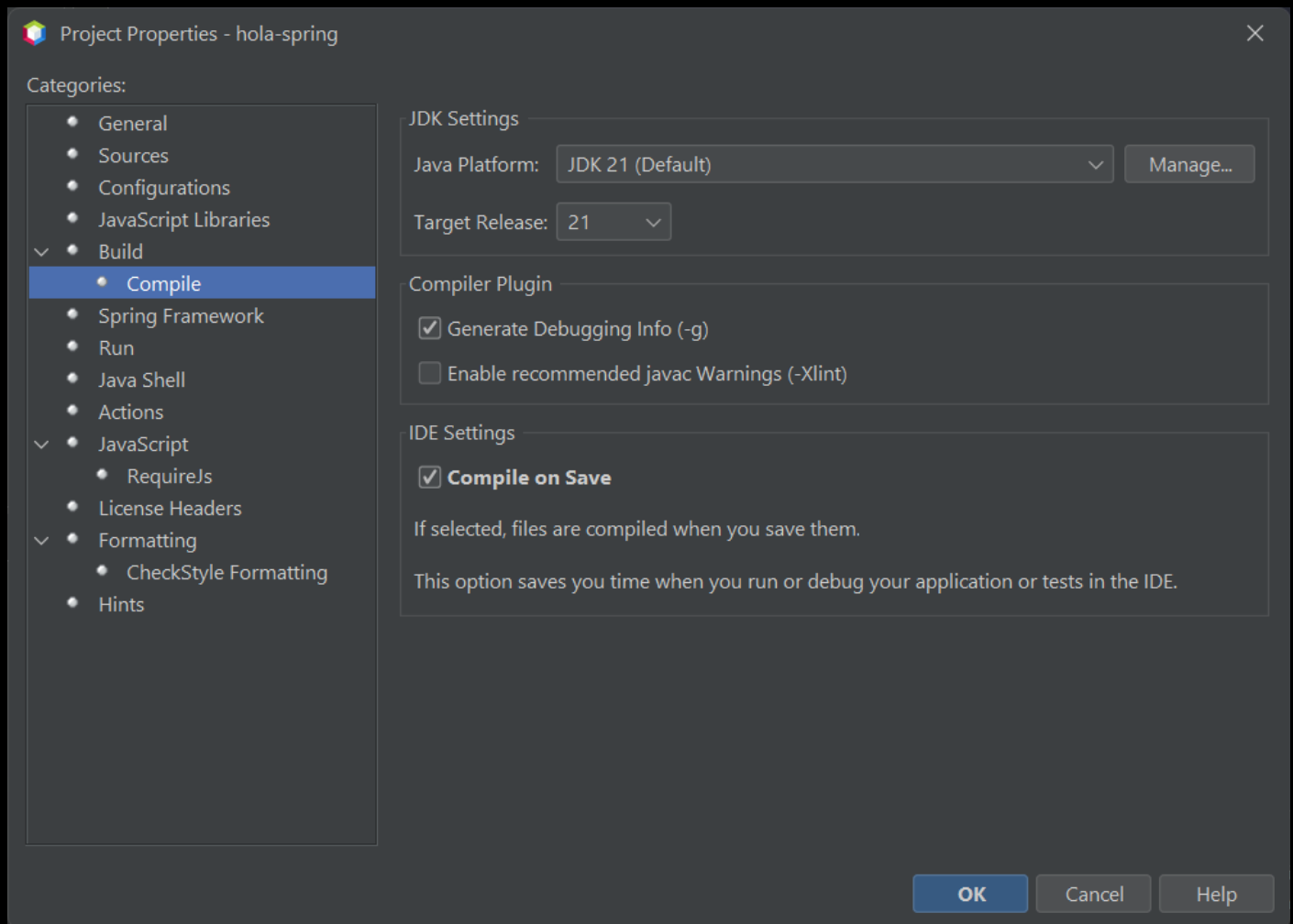
```
INFO Ejecutando el controlador REST
DEBUG Más detalle del controlador
```

(El mensaje `DEBUG` solo aparece si configuramos el nivel de logging)

Paso 6: Compilar al salvar

Para que devtools reinicia la aplicación de manera automática, es necesario configurar el proyecto de Netbeans para que se compile una vez guardado los cambios.

1. Dar click derecho sobre el proyecto y seleccionar Properties
2. Despues ir a Build > Compile y seleccionar la opción de Compile on Save
3. Con esto cualquier modificación sobre las clases Java se recargará en automática la aplicación.



Paso 7: Volver a Ejecutar y Probar

1. Puedes probar haciendo cambios en el texto del controlador, ej.

```
return "HolaMundo con Spring Boot 2";
```

2. Detén la ejecución actual de la aplicación (si ya está configurada la opción de Compile on Save se debería reiniciar de manera automática)
3. Haz clic derecho en `HolaSpringApplication.java` > `Run File` otra vez. (opcional si ya se reinició la aplicación de manera automática)
4. Ahora accede en tu navegador a:

```
http://localhost:9090
```

5. Verás el mensaje actualizado.

Conclusión

¡Listo! 🎉 Ahora tienes:

- Un controlador REST funcionando.
- Configuración personalizada de Spring Boot.
- Logging detallado para tu aplicación.

Sigue adelante con tu aprendizaje 🚀, ¡el esfuerzo vale la pena!

¡Saludos! 🙌

Ing. Marcela Gamiño e Ing. Ubaldo Acosta

Fundadores de [GlobalMentoring.com.mx](https://www.globalmentoring.com.mx)